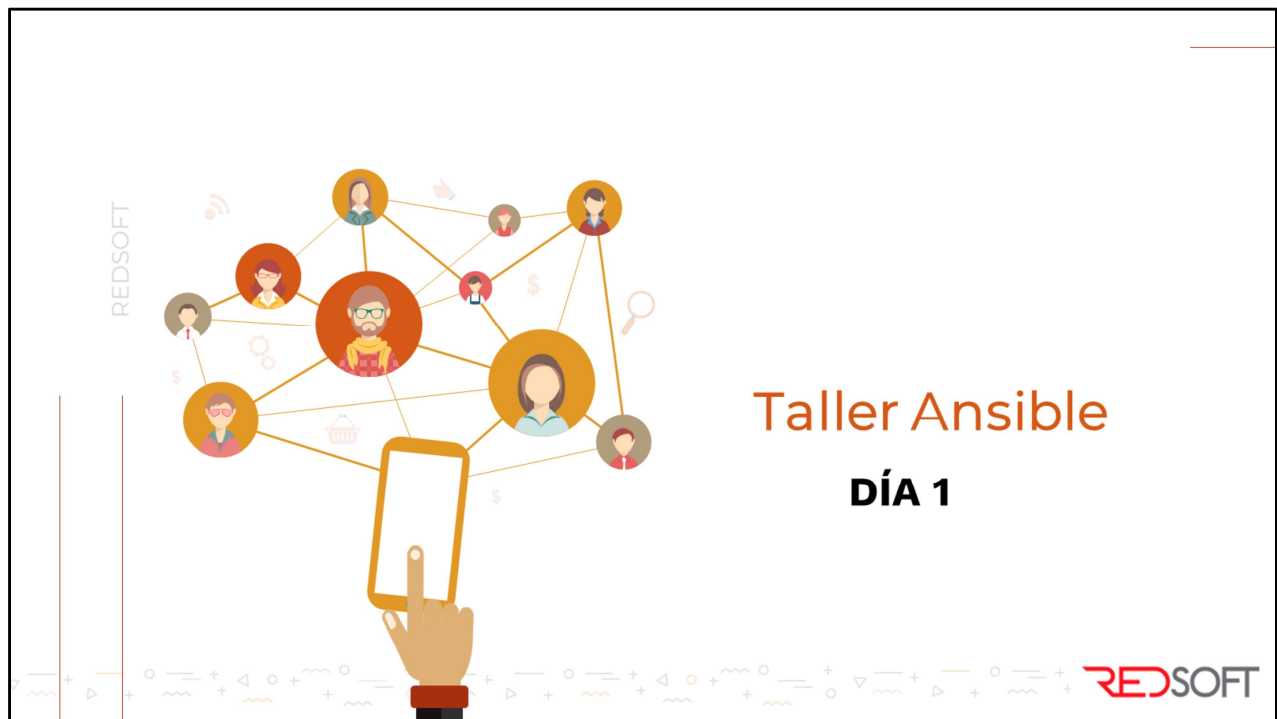




Automatización y administración de sistemas Linux

Durante muchos años, la mayor parte de la administración de sistemas y de infraestructuras se ha basado en tareas manuales realizadas a través de interfaces de usuario tanto gráficas como de línea de comandos. Los administradores de sistemas suelen usar listas de verificación, otra documentación o una rutina memorizada para realizar tareas estándares.

Este enfoque es propenso a errores. Es fácil que un administrador de sistemas omita un paso o cometa un error en un paso. A menudo, se hace una verificación limitada para comprobar que los pasos se hayan realizado correctamente o que produzcan el resultado esperado.

Además, al administrar cada servidor de forma manual e independiente, es muy fácil que muchos servidores que se supone que deben tener una configuración idéntica tengan diferencias menores (o importantes). Esto puede dificultar el mantenimiento e introducir errores o inestabilidad en el entorno de TI.

La automatización puede ayudar a evitar los problemas causados por la administración manual del sistema y la administración de la infraestructura. Como administrador de sistemas, puede usar la automatización para garantizar que todos sus sistemas se implementen y configuren de forma rápida y correcta. Por consiguiente, puede automatizar las tareas repetitivas de su cronograma diario a fin de tener más tiempo para enfocarse en tareas más importantes. Para su organización, la automatización puede ayudarlo a implementar más rápidamente

la próxima versión de una aplicación o las actualizaciones de un servicio.

Ansible



Ansible es una plataforma de automatización de código abierto. Es un *lenguaje de automatización simple* que puede describir de manera correcta una infraestructura de aplicaciones de TI en los playbooks de Ansible. También es un *motor de automatización* que ejecuta playbooks de Ansible.

Ansible puede administrar tareas de automatización potentes y puede adaptarse a muchos flujos de trabajo y entornos diferentes. Al mismo tiempo, los nuevos usuarios de Ansible pueden usarla muy rápidamente para volverse productivos

Ansible



- Simple
- Potente
- Sin agentes



Ansible es simple

Los playbooks de Ansible proporcionan automatización legible por el ojo humano. Esto significa que los playbooks son herramientas de automatización que también son fáciles de leer, comprender y cambiar para los humanos. No se necesitan habilidades de codificación especiales para escribirlas. Los playbooks ejecutan tareas en orden. La simplicidad del diseño del playbook hace que lo usen todos los equipos, lo que permite que las personas que nunca usaron Ansible se vuelvan productivas rápidamente.

Ansible es potente

Puede usar Ansible para implementar aplicaciones para la administración de la configuración, la automatización de flujos de trabajo y la automatización de redes. Puede usar Ansible para orquestar el ciclo de vida completo de la aplicación.

Ansible no tiene agentes

Ansible está diseñada alrededor de una arquitectura sin agentes. En general, Ansible se conecta con hosts que administra con OpenSSH o WinRM y ejecuta tareas, a menudo (pero no siempre) enviando pequeños programas denominados módulos Ansible a esos hosts. Estos programas se usan para colocar el sistema en un estado deseado específico. Todos los módulos enviados se eliminan cuando Ansible haya finalizado sus tareas. Puede comenzar a usar Ansible casi de forma inmediata, ya que no se necesita aprobar agentes especiales para usar y, luego, implementarlos en los hosts gestionados. Dado que no hay agentes ni una

infraestructura de seguridad personalizada adicional, Ansible es más eficaz y más segura que otras alternativas.

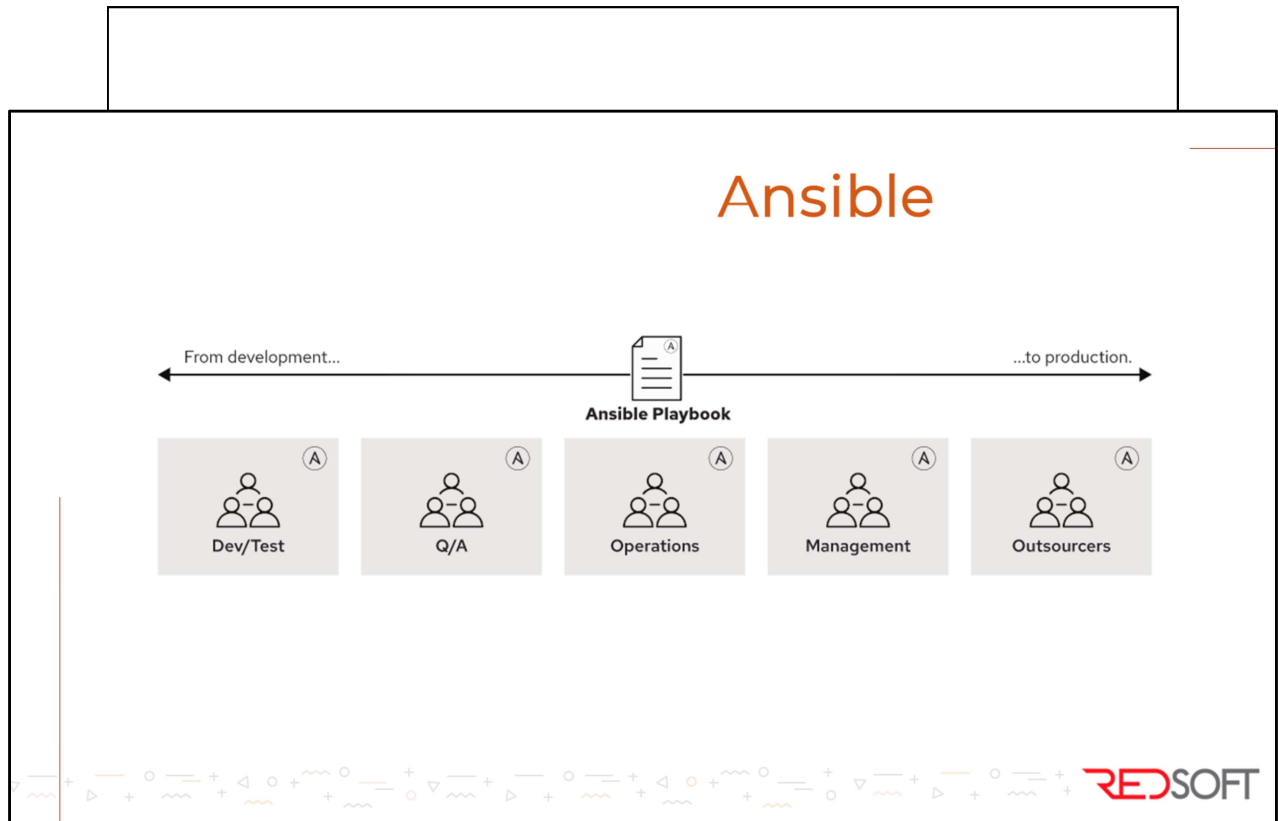
Ansible



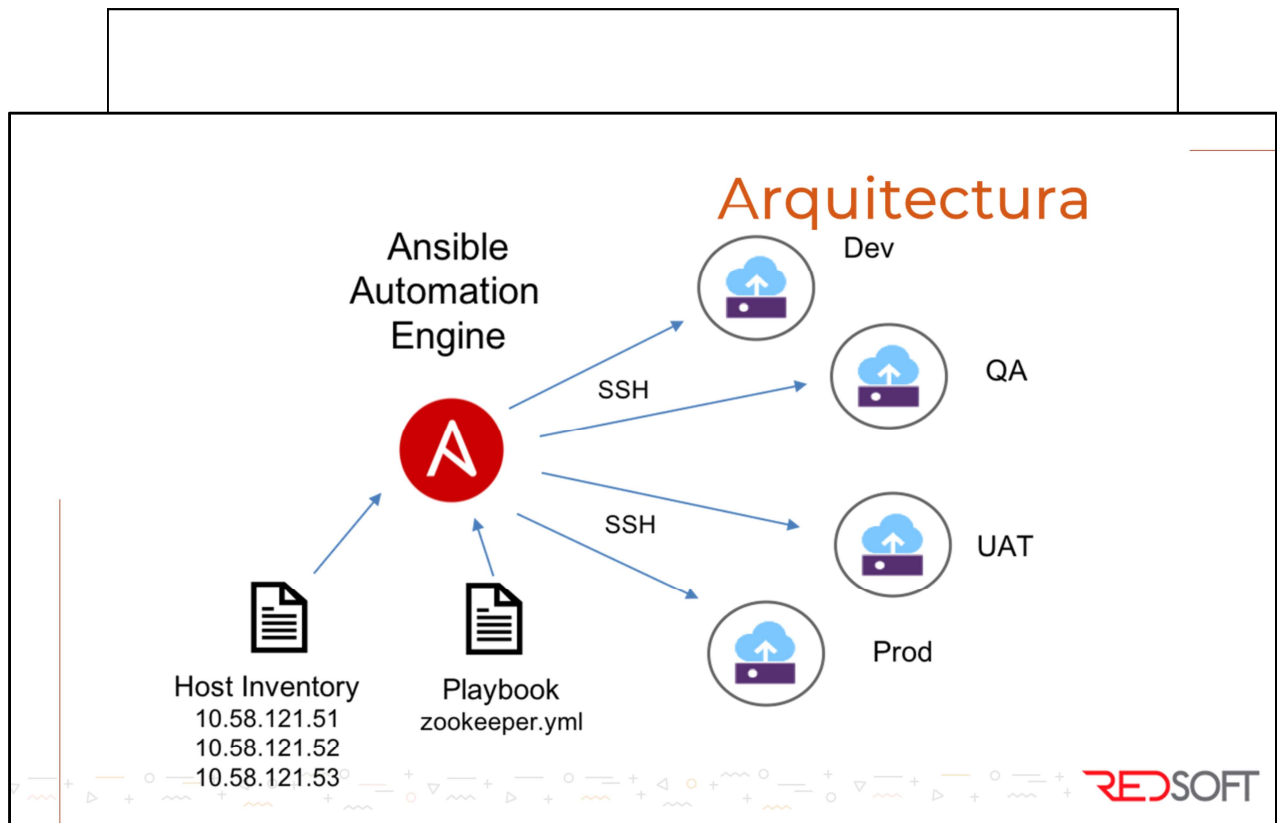
Puntos fuerte importantes:

- Soporte para diversas plataformas
- Automatización legible por el ojo humano
- Descripciones precisas de aplicaciones
- Fácil de administrar en el control de versiones
- Soporte para inventarios dinámicos
- Orquestación que se integra fácilmente con otros sistemas

- **Soporte para diversas plataformas:** Ansible proporciona soporte sin agentes para Linux, Windows, UNIX y dispositivos de red, en entornos físicos, virtuales, de nube y en contenedores.
- **Automatización legible por el ojo humano:** los playbooks de Ansible, escritos como archivos de texto YAML, son fáciles de leer y permiten garantizar que todos entiendan lo que harán.
- **Descripciones precisas de aplicaciones:** los playbooks de Ansible pueden realizar todas las modificaciones, y se pueden describir y documentar todos los aspectos de su entorno de aplicaciones.
- **Fácil de administrar en el control de versiones: los playbooks de Ansible y los proyectos son texto sin formato.** Puede considerarse código fuente y colocarse en su sistema de control de versiones existente.
- **Soporte para inventarios dinámicos:** la lista de máquinas que administra Ansible puede actualizarse de forma dinámica desde fuentes externas para capturar la lista actual y correcta de todos los servidores gestionados todo el tiempo, independientemente de la infraestructura o la ubicación.
- **Orquestación que se integra fácilmente con otros sistemas:** HP SA, Puppet, Jenkins, Red Hat Satellite, y otros sistemas que existen en su entorno, se pueden aprovechar e integrar en su workflow de Ansible.



La comunicación es la clave de DevOps. Ansible es el primer lenguaje de automatización que se puede leer y escribir en TI.

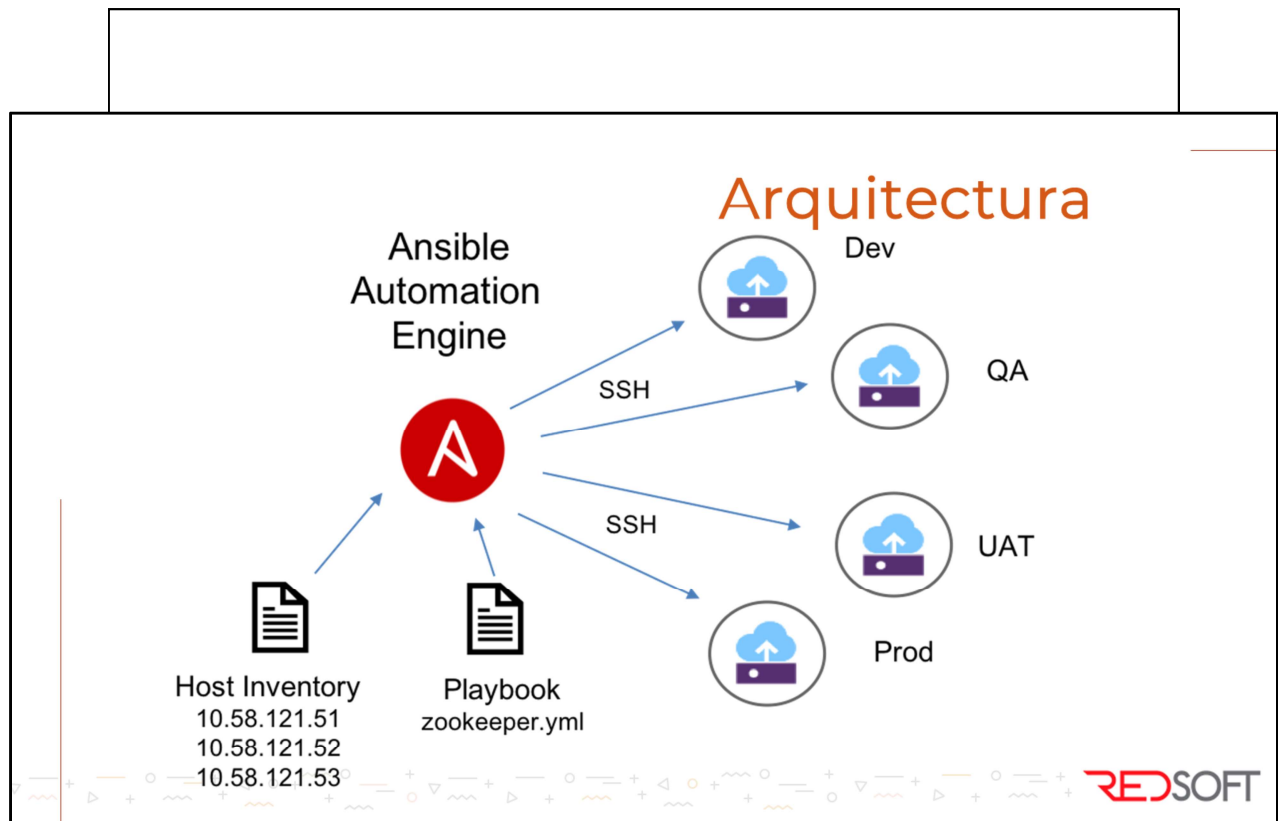


Hay dos tipos de máquinas en la arquitectura Ansible: nodos de control y hosts gestionados. Ansible se instala y se ejecuta desde un nodo de control, y esta máquina también tiene copias de sus archivos de proyecto de Ansible.

Los hosts gestionados aparecen en un inventario, que también organiza estos sistemas en grupos para una administración colectiva más sencilla. Puede definir el inventario de manera estática en un archivo de texto o de manera dinámica mediante scripts que obtienen información de grupos y host de fuentes externas.

En lugar de escribir scripts complejos, los usuarios de Ansible crean plays de alto nivel para asegurarse de que un host o grupo de hosts estén en un estado particular. Un play realiza una serie de tareas en los hosts, en el orden especificado por el play. Estas plays se expresan en formato YAML en un archivo de texto. Un archivo que contiene uno o más plays se denomina playbook.

Cada tarea ejecuta un módulo, una pequeña parte de un código (escrito en Python, PowerShell o cualquier otro lenguaje), con argumentos específicos. Cada módulo es esencialmente una herramienta en su kit de herramientas. Ansible se envía con cientos de módulos útiles que pueden realizar una amplia variedad de tareas de automatización. Pueden actuar en archivos de sistemas, instalar software o realizar llamadas de API.



Cuando se usa en una tarea, un módulo generalmente garantiza que un aspecto específico de la máquina tenga un estado específico. Por ejemplo, una tarea que usa un módulo puede garantizar que un archivo exista y tenga permisos y contenido específicos. Una tarea que usa un módulo diferente puede garantizar que se monte un sistema de archivos en particular. Si el sistema no tiene ese estado, la tarea debe colocarlo en ese estado o no hacer nada. Si una tarea falla, el comportamiento predeterminado de Ansible es cancelar el resto del playbook para los hosts que tuvieron una falla y continuar con los host restantes.

Las tareas, los plays y playbooks están diseñados para ser idempotentes. Esto significa que puede ejecutar de forma segura un playbook en los mismos hosts varias veces. Cuando sus sistemas tienen el estado correcto, el playbook no realiza cambios cuando lo ejecuta. Hay múltiples módulos disponibles que puede usar para ejecutar comandos arbitrarios. Sin embargo, debe usar esos módulos con cuidado para asegurarse de que se ejecuten de forma idempotente.

Ansible también usa complementos (plug-ins). Los complementos (plug-ins) son códigos que puede agregar a Ansible para ampliarla y adaptarla a los nuevos usos y las nuevas plataformas.

La arquitectura de Ansible no tiene agentes. En general, cuando un administrador ejecuta un playbook de Ansible, el nodo de control se conecta con el host gestionado mediante SSH (de forma predeterminada) o WinRM. Esto significa que

no necesita tener un agente específico de Ansible instalado en los hosts gestionados y no necesita permitir ninguna comunicación adicional entre el nodo de control y los hosts gestionados.

YAML

```
YAML:  
- Y: YAML  
- A: Ain't  
- M: Markup  
- L: Language
```

YAML es un lenguaje de serialización de datos legible por humanos que se utiliza a menudo para escribir archivos de configuración. Según a quién le preguntes, YAML significa yet another markup language (otro lenguaje de marcado más) o YAML ain't markup language (YAML no es un lenguaje de marcado) (un acrónimo recursivo), lo que enfatiza que YAML es para datos, no para documentos.

YAML es un lenguaje de programación popular porque está diseñado para ser fácil de leer y comprender. También se puede utilizar junto con otros lenguajes de programación. Debido a su flexibilidad y accesibilidad, Ansible® utiliza YAML para crear procesos de automatización, en forma de Ansible Playbooks.

YAML Syntax

```
# Strings don't require quotes:
title: Introduction to YAML

# But you can still use them:
title-w-quotes: 'Introduction to YAML'

# Multiline strings start with |
execute: |
  npm ci
  npm build
  npm test
```

```
# Integers:
age: 29

# Float:
price: 15.99

# Scientific notation:
population: 2.89e+6
```

Los archivos YAML usan una extensión .yaml o .yml y siguen reglas de sintaxis específicas.

YAML tiene características que provienen de Perl, C, XML, HTML y otros lenguajes de programación. YAML también es un superconjunto de JSON, por lo que los archivos JSON son válidos en YAML.

No hay símbolos de formato habituales, como llaves, corchetes, etiquetas de cierre o comillas. Y los archivos YAML son más simples de leer, ya que usan sangría al estilo Python para determinar la estructura e indicar anidamiento. Los caracteres de tabulación no están permitidos por diseño, para mantener la portabilidad entre sistemas, por lo que se usan espacios en blanco (caracteres de espacio literales) en su lugar.

Los comentarios se pueden identificar con un símbolo de almohadilla (#). Siempre es una buena práctica usar comentarios, ya que describen la intención del código. YAML no admite comentarios de varias líneas, cada línea debe tener como sufijo el carácter de almohadilla.

YAML

```
# Boolean values can be written in different ways:  
published: false  
published: False  
published: FALSE
```

```
date: 2002-12-14  
canonical: 2001-12-15T02:59:43.1Z  
iso8601: 2001-12-14t21:59:43.10-05:00  
spaced: 2001-12-14 21:59:43.10 -5
```



YAML

```
# A list of numbers using hyphens:
numbers:
  - one
  - two
  - three

# The inline version:
numbers: [ one, two, three ]
```

```
nineteen-eighty-four:
  author: George Orwell
  published-at: 1949-06-08
  page-count: 328
  description: |
    A Novel, often published as 1984, is a dystopian novel by English novelist George Orwell.
    It was published in June 1949 by Secker & Warburg as Orwell's ninth and final book.
```



YAML

```
- nineteen-eighty-four:
  author: George Orwell
  published-at: 1949-06-08
  page-count: 328
  description: |
    A Novel, often published as 1984, is a dystopian novel by English novelist George Orwell.

- the-hobbit:
  author: J. R. R. Tolkien
  published-at: 1937-09-21
  page-count: 310
  description: |
    The Hobbit, or There and Back Again is a children's fantasy novel by English author J. R. R. Tolkien.
```

```
# This is a really useful comment.
```



Inventario

Host name	Groups	Nested Groups
server1.tekneed.com	manchester	Europe
server2.tekneed.com	manchester	
server3.tekneed.com	barcelona	Europe
server4.tekneed.com	barcelona	
server5.tekneed.com	NIL	NIL
server6.tekneed.com	juve	Europe
server7.tekneed.com	juve	
server8.tekneed.com	NIL	NIL

Un *inventario* define un conjunto de hosts que gestiona Ansible. Estos hosts también pueden ser asignados a *grupos*, que pueden ser gestionados colectivamente. Los grupos pueden contener grupos secundarios, y los hosts pueden ser miembros de múltiples grupos. El inventario también puede establecer variables que se apliquen a los hosts y grupos que define.

Los inventarios de host de Ansible pueden incluir grupos de grupos de hosts. Esto se logra creando un nombre de grupo de host con el sufijo `:children`.

Inventario

```
server1.tekneed.com  
server2.tekneed.com  
server3.tekneed.com  
server4.tekneed.com  
server5.tekneed.com  
server6.tekneed.com  
server7.tekneed.com  
server8.tekneed.com
```

```
[manchester]  
server1.tekneed.com  
server2.tekneed.com  
  
[barcelona]  
server3.tekneed.com  
server4.tekneed.com  
  
[NIL]  
server5.tekneed.com  
server8.tekneed.com  
  
[juve]  
server6.tekneed.com  
server7.tekneed.com
```

```
[Europe:children]  
manchester  
barcelona  
juve
```

1. En su forma más simple, un archivo de inventario estático estilo INI es una lista de nombres de host o direcciones IP de hosts gestionados, cada uno en una línea única:
2. Sin embargo, normalmente organiza hosts gestionados en *grupos de hosts*. Los grupos de hosts le permiten ejecutar Ansible de manera más efectiva en comparación con un conjunto de sistemas. En este caso, cada sección comienza con un nombre de grupo de hosts encerrado entre corchetes ([]). A continuación, siguen el nombre de host o una dirección IP para cada host gestionado en el grupo, cada uno en una línea única.
3. Los hosts pueden estar en múltiples grupos. De hecho, la práctica recomendada es organizar sus hosts en múltiples grupos, posiblemente organizados de diferentes maneras, según el rol del host, su ubicación física, ya sea en producción o no, y otras variables. Esto le permite aplicar fácilmente plays de Ansible a conjuntos específicos de hosts en función de sus características, propósito o ubicación.
4. Los inventarios de host de Ansible pueden incluir grupos de grupos de hosts. Esto se logra creando un nombre de grupo de host con el sufijo :children.

Inventario

Simplificación de las especificaciones de hosts con rango

[START:END]

Ejemplos:

192.168.[4:7].[0:255] coincide con todas las direcciones IPv4 en la red 192.168.4.0/22 (de 192.168.4.0 a 192.168.7.255).

server[01:20].example.com coincide con todos los hosts denominados server01.example.com a server20.example.com.

[a:c].dns.example.com coincide con los hosts denominados a.dns.example.com, b.dns.example.com y c.dns.example.com.

2001:db8::[a:f] coincide con todas las direcciones IPv6 de 2001:db8::a a 2001:db8::f.



1. Puede especificar rangos en los nombres de hosts o direcciones IP para simplificar inventarios de hosts de Ansible. Puede especificar rangos numéricos o alfabéticos. Los rangos tienen la siguiente sintaxis:

Inventario

Verificación de hosts

```
[ansible@engine ~]$ ansible-inventory -i inventory --list
{
  "meta": {
    "hostvars": {}
  },
  "all": {
    "children": [
      "ungrouped",
      "usa",
      "canada"
    ]
  },
  "canada": {
    "hosts": [
      "ontario01.example.com",
      "ontario02.example.com"
    ]
  },
  "usa": {
    "hosts": [
      "washington1.example.com",
      "washington2.example.com"
    ]
  }
}
```

```
[ansible@engine ~]$ ansible-inventory -i inventory --graph
@all:
|--@ungrouped:
|--@usa:
| |--washington1.example.com
| |--washington2.example.com
|--@canada:
| |--ontario01.example.com
| |--ontario02.example.com
```

1. Puede especificar rangos en los nombres de hosts o direcciones IP para simplificar inventarios de hosts de Ansible. Puede especificar rangos numéricos o alfabéticos. Los rangos tienen la siguiente sintaxis:

Ansible

ansible.cfg

El comportamiento de una instalación de Ansible se puede personalizar modificando la configuración en el archivo de configuración de Ansible. Ansible elige su archivo de configuración de una de varias ubicaciones posibles en el nodo de control.

1. \$ANSIBLE_CONFIG

2. ./ansible.cfg

3. ~/.ansible.cfg

4. /etc/ansible/ansible.cfg

RED SOFT

Cree un archivo de configuración `ansible.cfg` en cada uno de sus directorios de proyectos de Ansible. El archivo de configuración de Ansible consiste en varias secciones; cada sección contiene parámetros definidos como pares de clave-valor. Los títulos de las secciones se incluyen entre paréntesis. Para la operación básica, use las siguientes dos secciones:

Ansible

```
[user@controlnode ~]$ ansible --version
ansible 2.9.21
  config file = /etc/ansible/ansible.cfg
  ...output omitted...
```

```
[defaults]
inventory = ./inventory
remote_user = user
ask_pass = false

[privilege_escalation]
become = true
become_method = sudo
become_user = root
become_ask_pass = false
```

- Para ver la ubicación del archivo de configuración se ejecuta el commando `ansible --version`
- `[defaults]`, que establece valores predeterminados para la operación de Ansible
- `[privilege_escalation]`, que configura el modo en que Ansible realiza el aumento de privilegios en hosts gestionados

Ansible

```
[user@controlnode ~]$ ansible --version
ansible 2.9.21
  config file = /etc/ansible/ansible.cfg
  ...output omitted...
```

```
[defaults]
inventory = ./inventory
remote_user = user
ask_pass = false

[privilege_escalation]
become = true
become_method = sudo
become_user = root
become_ask_pass = false
```

1. El parámetro `inventory` especifica la ruta a un archivo de inventario estático o a un directorio que contenga múltiples archivos de inventario estático y scripts de inventario dinámico.
2. El parámetro `remote_user` especifica el nombre de usuario que Ansible usa para conectarse a los hosts gestionados. En un entorno de ejecución de automatización basado en contenedores ejecutado por `ansible-navigator`, esto es `root` de forma predeterminada.
3. El parámetro `ask_pass` especifica si se debe solicitar o no una contraseña SSH. Establece `false` de forma predeterminada. Establezca este parámetro en `true` para la autenticación SSH basada en contraseña, o en `false` para la autenticación de clave pública SSH. Si usa el comando `ansible-navigator`, establecer este parámetro en `true` requiere deshabilitar los artefactos del `playbook` y usar el modo de salida estándar.
4. El parámetro `become` especifica si se debe cambiar automáticamente el usuario en el host gestionado (por lo general, a `root`) después de la conexión. Establece `false` de forma predeterminada. Si bien puede establecer este parámetro en `true` para habilitar el aumento de privilegios para todos los `plays`, puede decidir habilitar el aumento de privilegios solo en los `plays`, bloques o tareas que requieren privilegios elevados.
5. El parámetro `become_method` especifica cómo cambiar de usuario. El valor predeterminado es `sudo`, aunque hay otros métodos disponibles, como `su`.
6. El parámetro `become_user` especifica a qué usuario cambiar en el host gestionado. Establece `root` de forma predeterminada.

7. El parámetro `become_ask_pass` especifica si se debe solicitar o no una contraseña para el parámetro `become_method`. Establece `false` de forma predeterminada. Si usa el comando `ansible-navigator`, establecer este parámetro en `true` requiere deshabilitar los artefactos del playbook y usar el modo de salida estándar.

Ansible

Puntos a tener en cuenta con la escalación de privilegios:

1. Algunas tareas pueden completarse si el usuario tiene privilegios elevados.
2. Si sus playbooks utilizan un usuario regular se debe habilitar el aumento de privilegio
3. El archivo `/etc/sudoers` garantiza que todos los usuarios en el grupo `wheel` tengan capacidad de `sudo` de manera predeterminada
4. Cree un archivo en el directorio `/etc/sudoers.d` y asegúrese de que el archivo sea propiedad de `root`.
5. Si sus usuarios y grupos deben ingresar una contraseña para los comandos `sudo`, establezca el parámetro `become_ask_pass` en `true` en la sección `[privilege_escalation]` de su archivo de configuración de Ansible

```
## Allows people in group wheel to run all commands
%wheel  ALL=(ALL)    ALL

## Same thing without a password
# %wheel    ALL=(ALL)    NOPASSWD: ALL
```



Ansible

Configuración Conexión

De forma predeterminada, Ansible se conecta a los hosts administrados mediante el protocolo SSH.

```
[user@controlnode ~]$ ssh-copy-id root@web1.example.com
The authenticity of host 'web1.example.com (192.168.122.181)' can't be established.
ECDSA key fingerprint is 70:9c:03:cd:de:ba:2f:11:98:fa:a0:b3:7c:40:86:4b.
Are you sure you want to continue connecting (yes/no)? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
root@web1.example.com's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'root@web1.example.com'"
and check to make sure that only the key(s) you wanted were added.
```

Para ajustar este comportamiento y otras configuraciones de conexión, actualice los parámetros en la sección [defaults] del archivo de configuración de Ansible.

Establezca el parámetro `remote_user` para conectarse a sus hosts gestionados con un usuario específico. Si el parámetro `remote_user` no está definido, Ansible intenta conectarse con el usuario `root`

Puede configurar la autenticación SSH basada en claves; para ello, genere un par de claves SSH y, luego, copie la clave pública en sus hosts gestionados:

Si aún no tiene un par de claves SSH, ejecute el comando `ssh-keygen` para generar un par de claves SSH:

Use el comando `ssh-copy-id` para copiar su clave pública en un host gestionado. Por ejemplo, si su archivo `ansible.cfg` usa el parámetro `remote_user = root` y usted desea copiar su clave pública en el host `web.example.com`, ejecute el siguiente comando:

Ansible

Privilegios

Por razones de seguridad y auditoría, es posible que Ansible deba conectarse a hosts remotos como un usuario sin privilegios antes de escalar los privilegios para obtener acceso administrativo como root.

```
[defaults]
inventory = ./inventory
remote_user = someuser
ask_pass = false

[privilege_escalation]
become = true
become_method = sudo
become_user = root
become_ask_pass = false
```

- El parámetro `inventory` especifica la ruta a un archivo de inventario estático o a un directorio que contenga múltiples archivos de inventario estático y scripts de inventario dinámico.
- El parámetro `remote_user` especifica el nombre de usuario que Ansible usa para conectarse a los hosts gestionados. En un entorno de ejecución de automatización basado en contenedores ejecutado por `ansible-navigator`, esto es `root` de forma predeterminada.
- El parámetro `ask_pass` especifica si se debe solicitar o no una contraseña SSH. Establece `false` de forma predeterminada. Establezca este parámetro en `true` para la autenticación SSH basada en contraseña, o en `false` para la autenticación de clave pública SSH.
- El parámetro `become` especifica si se debe cambiar automáticamente el usuario en el host gestionado (por lo general, a `root`) después de la conexión. Establece `false` de forma predeterminada. Si bien puede establecer este parámetro en `true` para habilitar el aumento de privilegios para todos los plays, puede decidir habilitar el aumento de privilegios solo en los plays, bloques o tareas que requieren privilegios elevados.
- El parámetro `become_method` especifica cómo cambiar de usuario. El valor predeterminado es `sudo`, aunque hay otros métodos disponibles, como `su`.
- El parámetro `become_user` especifica a qué usuario cambiar en el host gestionado. Establece `root` de forma predeterminada.
- El parámetro `become_ask_pass` especifica si se debe solicitar o no una

contraseña para el parámetro `become_method`. Establece false de forma predeterminada.

Ansible

Módulos

Los módulos de Ansible son unidades de código que pueden controlar los recursos del sistema o ejecutar comandos del sistema.

Ansible proporciona una biblioteca de módulos que puede ejecutar directamente en hosts remotos o a través de playbooks. También puede escribir módulos personalizados.



Ansible

Módulos

Categoría	Los módulos
Archivos	• <code>ansible.builtin.copy</code>: copiar un archivo local en el host gestionado. • <code>ansible.builtin.file</code>: establecer permisos y otras propiedades de archivos. • <code>ansible.builtin.lineinfile</code>: garantizar que una línea particular esté o no esté en un archivo. • <code>ansible.posix.synchronize</code>: sincronizar contenido usando <code>rsync</code>.
Software	• <code>ansible.builtin.package</code>: administrar los paquetes usando el gestor de paquetes detectado automáticamente nativo del sistema operativo. • <code>ansible.builtin.dnf</code>: administrar paquetes usando el administrador de paquetes DNF. • <code>ansible.builtin.apt</code>: administrar paquetes usando el administrador de paquetes APT. • <code>ansible.builtin.pip</code>: administrar paquetes de Python desde PyPI.
Sistema	• <code>ansible.posix.firewalld</code>: administrar puertos y servicios arbitrarios con <code>firewalld</code>. • <code>ansible.builtin.reboot</code>: reiniciar una máquina. • <code>ansible.builtin.service</code>: administrar servicios. • <code>ansible.builtin.user</code>: agregar, eliminar y administrar cuentas de usuario.
Herramientas de red	• <code>ansible.builtin.get_url</code>: descargar archivos por HTTP, HTTPS o FTP. • <code>ansible.builtin.uri</code>: interactuar con servicios web.

REDSOFT

Ansible

Módulos

https://docs.ansible.com/ansible/2.9/modules/modules_by_category.html

```
[user@controlnode ~]$ ansible-doc ping
> PING    (/usr/lib/python3.6/site-packages/ansible/modules/system/ping.py)

    A trivial test module, this module always returns 'pong' on successful contact. It does not make sense in playbooks, but it is useful from '/usr/bin/ansible' to
    verify the ability to login and that a usable Python is configured. This is NOT ICMP ping, this is just a trivial test module that requires Python on the
    remote-node. For Windows targets, use the [win_ping] module instead. For Network targets, use the [net_ping] module instead.

    * This module is maintained by The Ansible Core Team
    OPTIONS (= is mandatory):

    - data
      Data to return for the 'ping' return value.
      If this parameter is set to 'crash', the module will cause an exception.
      [Default: pong]
      type: str
```



Modulos

add_host	Add a host (and alternatively a group) to the ansible-playbo...
apt	Manages apt-packages
apt_key	Add or remove an apt key
apt_repository	Add and remove APT repositories
assemble	Assemble configuration files from fragments
assert	Asserts given expressions are true
async_status	Obtain status of asynchronous task
blockinfile	Insert/update/remove a text block surrounded by marker lines
command	Execute commands on targets
copy	Copy files to remote locations
cron	Manage cron.d and crontab entries
debconf	Configure a .deb package
debug	Print statements during execution
dnf	Manages packages with the 'dnf' package manager
dpkg_selections	Dpkg package selection selections
expect	Executes a command and responds to prompts
fail	Fail with custom message
fetch	Fetch files from remote nodes
file	Manage files and file properties
find	Return a list of files based on specific criteria
gather_facts	Gathers facts about remote hosts
get_url	Downloads files from HTTP, HTTPS, or FTP to node



Ansible-doc -l

Modulos

script	Runs a local script on a remote node after transferring it
service	Manage services
service_facts	Return service state information as fact data
set_fact	Set host variable(s) and fact(s)
set_stats	Define and display stats for the current ansible run
setup	Gathers facts about remote hosts
shell	Execute shell commands on targets
slurp	Slurps a file from remote nodes
stat	Retrieve file or file system status
subversion	Deploys a subversion repository
systemd	Manage systemd units
sysvinit	Manage SysV services
tempfile	Creates temporary files and directories
template	Template a file out to a target host
unarchive	Unpacks an archive after (optionally) copying it from the lo...
uri	Interacts with webservices
user	Manage user accounts
validate_argument_spec	Validate role argument specs
wait_for	Waits for a condition before continuing
wait_for_connection	Waits until remote system is reachable/usable
yum	Manages packages with the 'yum' package manager
yum_repository	Add or remove YUM repositories



Si no existe un módulo para automatizar alguna tarea, hay módulos especiales disponibles que pueden ejecutar comandos arbitrarios en sus hosts gestionados.

El módulo `ansible.builtin.command` es el más simple de estos comandos. Su argumento `cmd` especifica el comando que desea ejecutar.

Ansible

Módulos

```
[user@controlnode ~]$ ansible -m user -a 'name=newbie uid=4000 state=present' \
> servera.lab.example.com
servera.lab.example.com | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/libexec/platform-python"
  },
  "changed": true,
  "comment": "",
  "createhome": true,
  "group": 4000,
  "home": "/home/newbie",
  "name": "newbie",
  "shell": "/bin/bash",
  "state": "present",
  "system": false,
  "uid": 4000
}
```

```
[user@controlnode ~]$ ansible mymanagedhosts -m command -a /usr/bin/hostname
host1.lab.example.com | CHANGED | rc=0 >>
host1.lab.example.com
host2.lab.example.com | CHANGED | rc=0 >>
host2.lab.example.com
```

RED SOFT

`ansible all -m command -a 'ls /etc/yum.repos'`

`ansible all -m yum_repository -a 'file=external.repo name=BaseOS
description="Base OS Repo" baseurl=file:///mnt/BaseOS/ gpgcheck=yes
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release enabled=yes
state=present' -b`

Modulos

```
- name: Run the /opt/bin/makedb.sh command
ansible.builtin.command:
  cmd: /opt/bin/makedb.sh
```

```
- name: Non-idempotent approach with shell module
ansible.builtin.shell:
  cmd: echo "nameserver 192.0.2.1" > /etc/resolv.conf
```



El módulo `ansible.builtin.command` es el más simple de estos comandos. Su argumento `cmd` especifica el comando que desea ejecutar.

La siguiente tarea de ejemplo ejecuta `/opt/bin/makedb.sh` en hosts gestionados.

El módulo `ansible.builtin.command` no puede acceder a las variables del entorno de la shell ni realizar operaciones de la shell, como redirección de entrada/salida o pipelines. Cuando necesite realizar un procesamiento de shell, puede usar el

módulo `ansible.builtin.shell`. Al igual que el módulo `ansible.builtin.command`, pase los comandos que se deben ejecutar como argumentos al módulo.

Ambos módulos `ansible.builtin.command` y `ansible.builtin.shell` requieren una instalación Python de trabajo en el host gestionado. Un tercer módulo, `ansible.builtin.raw`, puede ejecutar comandos directamente con la shell remota y eludir el s

ubsistema del módulo. Esto es útil cuando administra sistemas que no pueden tener Python instalado (por ejemplo, un enrutador de red). También puede usarse para instalar Python en un host gestionado.

Ansible

Playbook

Los comandos ad hoc pueden ejecutar una tarea única y simple en un conjunto de hosts específicos como un comando de una sola vez.

Sin embargo, el poder real de Ansible está en aprender a usar playbook para ejecutar tareas múltiples y complejas contra un conjunto de hosts objetivo de una manera fácilmente repetible.



Ansible

Playbook

- Una tarea o task es la aplicación de un módulo para realizar una unidad de trabajo específica.
- Un play es una secuencia de tareas que se aplicarán, en orden, a uno o más hosts seleccionados de su inventario.
- Un playbook es un archivo de texto que contiene una lista de una o más plays para ejecutar en un orden específico.



Los plays le permiten modificar un conjunto complejo y extenso de tareas administrativas manuales en una rutina fácilmente repetible con resultados predecibles y satisfactorios. En un playbook, puede guardar la secuencia de tareas en un play de una forma inmediatamente ejecutable y legible por el ojo humano. Las tareas en sí, debido a la manera en la que se escriben, documentan los pasos necesarios para implementar su aplicación o infraestructura.

Ansible

Playbook

- Los elementos de datos del mismo nivel en la jerarquía (como los elementos de la misma lista) deben tener la misma sangría.
- Los elementos que son hijos de otro elemento deben tener más sangría que sus padres.

```
---  
- name: Configure important user consistently  
  hosts: servera.lab.example.com  
  tasks:  
    - name: newbie exists with UID 4000  
      user:  
        name: newbie  
        uid: 4000  
        state: present
```

REDSOFT

Un playbook es un archivo de texto escrito en formato YAML y se guarda generalmente con la extensión .yaml. El playbook usa sangría con caracteres de espacio para indicar la estructura de sus datos. YAML no establece requisitos estrictos sobre cuántos espacios se usan para la sangría, pero se aplican dos reglas básicas.

Si usa el editor de texto vi, puede aplicar algunos parámetros que pueden facilitar la edición de sus playbooks. Por ejemplo, puede agregar la siguiente línea a su archivo \$HOME/.vimrc y, cuando vi detecta que usted está editando un archivo YAML, realiza una sangría de dos espacios cuando presiona la tecla **Tab** y realiza una sangría automática de las líneas posteriores.

```
autocmd FileType yaml setlocal ai ts=2 sw=2 et
```

Ansible

Playbook

```
---  
- name: Configure important user consistently  
  hosts: servera.lab.example.com  
  tasks:  
    - name: newbie exists with UID 4000  
      user:  
        name: newbie  
        uid: 4000  
        state: present
```

El playbook comienza con una línea que consiste en tres guiones (---) para marcar el inicio de documento. También puede finalizar con tres puntos (...) como para indicar el fin del documento, si bien en la práctica por lo general se omite.

El play de ejemplo inicial tiene tres claves: name, hosts y tasks. Todas estas claves tienen la misma sangría.

La primera línea del play de ejemplo comienza con un guion y un espacio (que indican que el play es el primer ítem de una lista); luego, sigue la primera clave, name. La clave name asocia una cadena arbitraria con el play como una etiqueta que identifica el propósito del play. La clave name es opcional, pero se recomienda, ya que ayuda a documentar su playbook. Esto es especialmente útil cuando un playbook contiene múltiples plays.

La segunda clave en el play es una clave hosts, que especifica los hosts respecto de los cuales se deben ejecutar las tareas del play. La clave hosts toma un patrón de host como valor, como los nombres de los hosts gestionados o grupos en el inventario

Finalmente, la última clave en el play es tasks, cuyo valor especifica una lista de tareas para ejecutar para este play. Este ejemplo tiene una sola tarea que ejecuta el módulo `ansible.builtin.user` con argumentos específicos (para garantizar que el usuario `newbie` exista y tenga una UID de 4000).

La clave tasks es la parte del play que detalla realmente, en orden, las tareas que se ejecutarán en los hosts gestionados. Cada tarea en la lista es en sí una recopilación de pares de clave-valor.

Ansible

Playbook

```
tasks:
  - name: Web server is enabled
    ansible.builtin.service:
      name: httpd
      enabled: true

  - name: NTP server is enabled
    ansible.builtin.service:
      name: chronyd
      enabled: true

  - name: Postfix is enabled
    ansible.builtin.service:
      name: postfix
      enabled: true
```



Se incluye otro ejemplo de una clave tasks con varias tareas, que usan el módulo `ansible.builtin.service` para garantizar que un servicio

Ansible

Verificación y ejecución de Playbook

```
[ansible@pcansible ansible]$ ansible-playbook new_user.yml --syntax-check
[WARNING]: Could not match supplied host pattern, ignoring: servera.lab.example.com
playbook: new_user.yml
```

```
[ansible@pcansible ansible]$ ansible-playbook new_user.yml --syntax-check
playbook: new_user.yml
[ansible@pcansible ansible]$ ansible-playbook new_user.yml

PLAY [Configure important user consistently] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Newbie exists with UID 4000] *****
changed: [localhost]

PLAY RECAP *****
localhost : ok=2  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```



Ansible

Implementación de varios Playbooks

```
---
# This is a simple playbook with two plays

- name: First play
  hosts: web.example.com
  tasks:
    - name: First task
      ansible.builtin.dnf:
        name: httpd
        state: present

    - name: Second task
      ansible.builtin.service:
        name: httpd
        enabled: true

- name: Second play
  hosts: database.example.com
  tasks:
    - name: First task
      ansible.builtin.service:
        name: mariadb
        enabled: true
```

Ansible

Playbook – Variables

Ansible admite variables que se pueden usar para almacenar valores que luego se pueden reutilizar en los archivos de un proyecto de Ansible.

Esto puede simplificar la creación y el mantenimiento de un proyecto y reducir el número de errores.

Invalid variable names	Valid variable names
web server	web_server
remote.file	remote_file
1st file	file_1 file1
remoteserver\$1	remote_server_1 remote_server1

Las variables brindan una forma conveniente de gestionar los valores dinámicos para un entorno dado en su proyecto Ansible. Los ejemplos de valores que pueden contener las variables

incluyen:

Usuarios para crear

Paquetes para instalar

Servicios para reiniciar

Archivos para eliminar

Colección de archivos para recuperar de Internet

Denominación de variables

Los nombres de las variables deben iniciarse con una letra y solo pueden contener letras, números y guiones bajos.

Ansible

Playbook – Variables

La siguiente lista simplificada muestra formas de definir una variable, ordenada de menor a mayor precedencia:

- Variables de grupo definidas en el inventario
- Agrupar variables definidas en archivos en un subdirectorio group_vars en el mismo directorio que el inventario o el playbook
- Variables de host definidas en el inventario
- Variables de host definidas en archivos en un subdirectorio host_vars en el mismo directorio que el inventario o el playbook
- Datos del host, descubiertos en tiempo de ejecución
- Variables de play en el playbook (vars y vars_files).
- Variables de tareas
- Variables adicionales definidas en la línea de comandos



Las variables se pueden definir en una variedad de lugares en un proyecto Ansible. Si una variable se establece con el mismo nombre en dos lugares, y esas configuraciones tienen valores diferentes, la *precedencia* determina qué valor se usa.

Puede establecer una variable que afecte a un grupo de hosts o solo a hosts individuales. Algunas variables son *datos* que Ansible puede establecer en función de la configuración de un sistema. Se pueden establecer otras variables dentro del playbook y afectar a un play en ese playbook o solo a una tarea en ese play. También puede establecer *variables adicionales* en la línea de comandos ansible-navigator run con la opción --extra-vars o -e y especificar esas variables, que anulan todos los demás valores para ese nombre de variable.

Ansible

Playbook – Variables

Las variables juegan un papel importante en Ansible Playbooks porque facilitan la gestión de datos variables en un playbook.

```
- hosts: all
  vars:
    user: joe
    home: /home/joe
```

```
- hosts: all
  vars_files:
    - vars/users.yml
```

```
vars:
  user: joe

tasks:
  # This line will read: Creates the user joe
  - name: Creates the user {{ user }}
    user:
      # This line will create the user named Joe
      name: "{{ user }}"
```

REDSOFT

1. puede definir variables de playbook de múltiples maneras. Uno de los métodos más comunes es colocar una variable en un bloque vars al comienzo de un play
2. También es posible definir variables de playbooks en archivos externos. En este caso, en lugar de usar el bloque vars en el playbook, se puede usar la directiva vars_files, seguida de una lista de nombres para archivos de variables externas, relativos a la ubicación del playbook
3. Una vez declaradas las variables, puede usar las variables en tareas. Se hace referencia a las variables colocando el nombre de la variable entre llaves dobles ({{ }}). Ansible sustituye la variable con su valor cuando se ejecuta la tarea.

Ansible

Playbook – Variables

```
[servers]
demo.example.com  ansible_user=joe
```

```
[servers1]
demo1.example.com
demo2.example.com
```

```
[servers2]
demo3.example.com
demo4.example.com
```

```
[servers:children]
servers1
servers2
```

```
[servers:vars]
user=joe
```

RED SOFT

Las variables de inventario que se aplican directamente a los hosts se dividen en dos categorías amplias: *variables de host*, que se aplican a un host específico; y *variables de grupo*, que se aplican a todos los hosts en un grupo de hosts o en un grupo de grupos de hosts. Las variables de hosts preceden a las variables de grupos, pero las variables definidas por un playbook preceden a ambas.

Una forma de definir las variables de hosts y las variables de grupos es hacerlo directamente en el archivo de inventario.

Definición de la variable de host `ansible_user` para `demo.example.com`:

Definición de la variable de grupo `user` para el grupo `servers`, que consta de dos grupos de hosts, cada uno con dos servidores.

Ansible

Playbook – Variables

```
user1_first_name: Bob
user1_last_name: Jones
user1_home_dir: /users/bjones
user2_first_name: Anne
user2_last_name: Cook
user2_home_dir: /users/acook
```

```
users:
  bjones:
    first_name: Bob
    last_name: Jones
    home_dir: /users/bjones
  acook:
    first_name: Anne
    last_name: Cook
    home_dir: /users/acook
```



Uso de diccionarios como variables

En lugar de asignar datos de configuración que se relacionen con el mismo elemento con múltiples variables, los administradores pueden usar *diccionarios*. Un diccionario es una estructura de datos que contiene pares de clave-valor, donde los valores también pueden ser diccionarios.

Por ejemplo, considere el siguiente fragmento:

Este se puede reescribir como un diccionario denominado users:

Ansible

Playbook – Variables

```
# Returns 'Bob'  
users.bjones.first_name
```

```
# Returns '/users/acook'  
users.acook.home_dir
```

```
# Returns 'Bob'  
users['bjones']['first_name']
```

```
# Returns '/users/acook'  
users['acook']['home_dir']
```



Luego puede usar las siguientes variables para acceder a los datos del usuario:

Debido a que la variable se define como un diccionario Python, hay disponible una sintaxis alternativa.

Ansible

Playbook – Variables

```
---
- name: Installs a package and prints the result
  hosts: all
  tasks:
    - name: Install the package
      yum:
        name: httpd
        state: installed
        register: install_result
    - debug:
        var: install_result
```



Puede usar la instrucción `register` para capturar la salida de un comando u otra información sobre la ejecución de un módulo. La salida se guarda en una variable que puede usarse posteriormente en el playbook con motivos de depuración u otros motivos, como aplicar una configuración particular basada en la salida de un comando.

El siguiente play demuestra cómo capturar el resultado de un comando con motivos de depuración:

Ansible

Playbook – Variables

```
[user@demo ~]$ ansible-playbook playbook.yml
PLAY [Installs a package and prints the result] *****

TASK [setup] *****
ok: [demo.example.com]

TASK [Install the package] *****
ok: [demo.example.com]

TASK [debug] *****
ok: [demo.example.com] => {
  "install_result": {
    "changed": false,
    "msg": "",
    "rc": 0,
    "results": [
      "httpd-2.4.6-40.el7.x86_64 providing httpd is already installed"
    ]
  }
}

PLAY RECAP *****
demo.example.com : ok=3  changed=0  unreachable=0  failed=0
```

RED SOFT

Cuando se ejecuta el play, el módulo debug vuelca el valor de la variable registrada `install_result` en el terminal.

Facts

Facts

Los facts de Ansible son variables que Ansible descubre automáticamente en un host administrado.

Los facts contienen información específica del host que se puede usar como variables regulares en reproducciones, condicionales, bucles o cualquier otra declaración que dependa de un valor recopilado de un host administrado.

Normalmente, cada juego ejecuta el módulo de configuración automáticamente antes de la primera tarea para recopilar datos. Esto se informa como la tarea recopilación de datos en Ansible 2.3 y versiones posteriores, o simplemente como configuración en versiones anteriores de Ansible.



Facts

Facts – Ejemplos

- Hostname
- La versión del Kernel
- Las interfaces de red
- Las direcciones IP
- La versión del sistema operativo
- Varias variables de entorno
- El número de CPU
- La memoria disponible o libre
- El espacio disponible en disco

```
- name: Fact dump
hosts: all
tasks:
  - name: Print all facts
    debug:
      var: ansible_facts
```



```
[user@demo ~]$ ansible-playbook facts.yml

PLAY [Fact dump] *****

TASK [Gathering Facts] *****
ok: [demo1.example.com]

TASK [Print all facts] *****
ok: [demo1.example.com] => {
  "ansible_facts": {
    "all_ipv4_addresses": [
      "172.25.250.10"
    ],
    "all_ipv6_addresses": [
      "fe80::5054:ff:fe00:fa0a"
    ],
    "ansible_local": {},
    "apparmor": {
      "status": "disabled"
    },
    "architecture": "x86_64",
    "bios_date": "01/01/2011",
    "bios_version": "0.5.1",
    "cmdline": {
      "BOOT_IMAGE": "/boot/vmlinuz-3.10.0-327.el7.x86_64",
      "LANG": "en_US.UTF-8",
      "console": "ttyS0,115200n8",
      "crashkernel": "auto",
      "net.ifnames": "0",
      "no_timer_check": true,
      "ro": true,
      "root": "UUID=2460ab6e-e869-4011-acae-31b2e8c05a3b"
    }
  }
}
```

Facts



Cuando ejecuta el playbook, los datos se muestran en la salida del trabajo:

Facts

Facts – Ejemplos

Fact	Variable
Short host name	<code>ansible_facts['hostname']</code>
Fully qualified domain name	<code>ansible_facts['fqdn']</code>
Main IPv4 address (based on routing)	<code>ansible_facts['default_ipv4']['address']</code>
List of the names of all network interfaces	<code>ansible_facts['interfaces']</code>
Size of the <code>/dev/vda1</code> disk partition	<code>ansible_facts['devices']['vda']['partitions']['vda1']['size']</code>
List of DNS servers	<code>ansible_facts['dns']['nameservers']</code>
Version of the currently running kernel	<code>ansible_facts['kernel']</code>

En la siguiente tabla se muestran algunos de los datos que podrían recopilarse de un nodo administrado, los cuales pueden ser útiles en un playbook:

Facts

Facts – Ejemplos

```
---
- hosts: all
  tasks:
    - name: Prints various Ansible facts
      debug:
        msg: >
          The default IPv4 address of {{ ansible_facts.fqdn }}
          is {{ ansible_facts.default_ipv4.address }}
```

```
[user@demo ~]$ ansible-playbook playbook.yml
PLAY *****

TASK [Gathering Facts] *****
ok: [demo1.example.com]

TASK [Prints various Ansible facts] *****
ok: [demo1.example.com] => {
  "msg": "The default IPv4 address of demo1.example.com is
    172.25.250.10"
}

PLAY RECAP *****
demo1.example.com : ok=2    changed=0    unreachable=0    failed=0
```



Cuando un dato se usa en un playbook, Ansible sustituye de manera dinámica el nombre de la variable para el dato con el valor correspondiente:

En la siguiente salida, se muestra cómo Ansible consiguió consultar el nodo gestionado y usar de manera dinámica la información del sistema para actualizar la variable. También puede usar datos para crear grupos de hosts dinámicos que combinen criterios en particular.

Facts

Facts

Hay algunas razones por las que esto podría requerir deshabilitar la colección de facts, por ejemplo:

- Puede ser que no esté usando ningún dato y desee acelerar la play o reducir la carga causada por los play en los hosts administrados.
- Es posible que los hosts administrados no puedan ejecutar el módulo de configuración por algún motivo
- Se necesite instalar algún software de requisito previo antes de recopilar datos.

```
---  
- name: This play gathers no facts automatically  
  hosts: large_farm  
  gather_facts: no
```

Para deshabilitar la recopilación de datos para un play, establezca la palabra clave `gather_facts` en `false`:

Magic Variables

Magic Variables

Algunas variables no son facts ni se configuran a través del módulo de *setup*, pero Ansible también las configura automáticamente. Estas variables mágicas también pueden ser útiles para obtener información específica de un host administrado en particular.

- `hostvars`
- `group_names`
- `Groups`
- `Inventory_hostname`

Referencia

https://docs.ansible.com/ansible/2.9/user_guide/playbooks_variables.html#variable-precedence-where-should-i-put-a-variable



Hostvars: contiene las variables para hosts gestionados y puede usarse para obtener los valores para las variables de otro host gestionado. No incluye los datos del host gestionado si no se

recopilaron aún para ese host.

group_names: Enumera todos los grupos en los que se encuentra el host gestionado actual.

Groups: Enumera todos los grupos y hosts en el inventario.

inventory_hostname: Contiene el nombre de host para el host gestionado actual, según se configuró en el inventario. Este podría ser diferente del nombre de host informado por los datos por diversos motivos.

Magic Variables

```
[user@demo ~]$ ansible localhost -m debug -a 'var=hostvars["localhost"]'
localhost | SUCCESS => {
  "hostvars[\"localhost\"]": {
    "ansible_check_mode": false,
    "ansible_connection": "local",
    "ansible_diff_mode": false,
    "ansible_facts": {},
    "ansible_forks": 5,
    "ansible_inventory_sources": [
      "/home/student/demo/inventory"
    ],
    "ansible_playbook_python": "/usr/bin/python2",
    "ansible_python_interpreter": "/usr/bin/python2",
    "ansible_verbosity": 0,
    "ansible_version": {
      "full": "2.7.0",
      "major": 2,
      "minor": 7,
      "revision": 0,
      "string": "2.7.0"
    },
    "group_names": [],
    "groups": {
      "all": [
        "serverb.lab.example.com"
      ],
      "ungrouped": [],
      "webserver": [
        "serverb.lab.example.com"
      ]
    },
    "inventory_hostname": "localhost",
    "inventory_hostname_short": "localhost",
    "omit": "_omit_place_holder__18d132963728b2cbf7143dd49dc4bf5745fe5ec3",
    "playbook_dir": "/home/student/demo"
  }
}
```

hostvars

Contiene las variables de los hosts administrados y se puede utilizar para obtener los valores de las variables de otro host administrado. No incluye los datos del host administrado si aún no se han recopilado para ese host.

group_names

Enumera todos los grupos en los que se encuentra el host administrado actual.

groups

Enumera todos los grupos y hosts del inventario.

inventory_hostname

Contiene el nombre de host del host administrado actual tal como está configurado en el inventario. Este puede ser diferente del nombre de host informado por los datos por varias razones.

Preguntas / Respuestas



